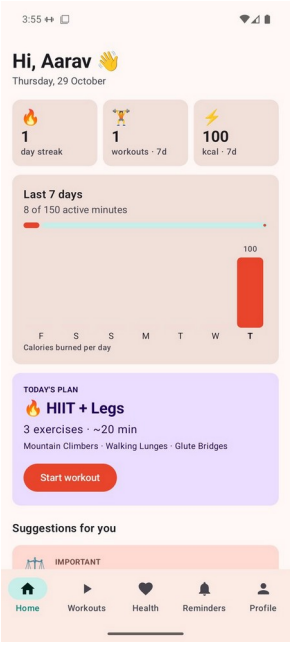
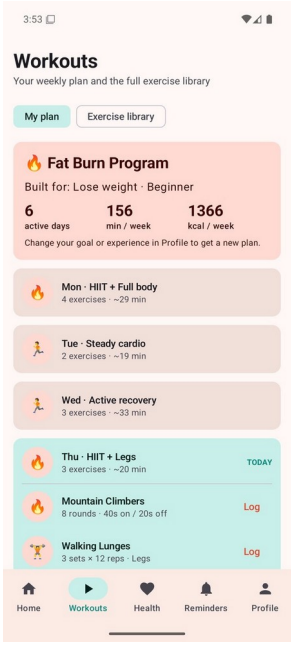
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY VADODARA
International Campus Diu · Department of Computer Science and Engineering

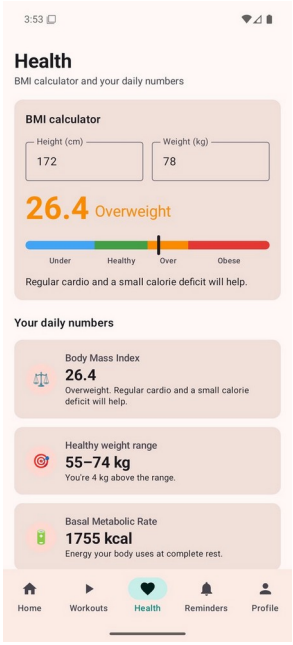
OBJECT-ORIENTED PROGRAMMING · PROJECT REPORT

FitMate

A cross-platform fitness and workout trainer built with object-oriented design







SUBMITTED BY	ROLL NUMBER
Shivang Raj	20252501076
Bhumika Gupta	20252501095
Vir Sharma	20252501093
Deepansh Singh	20252501019

Source code: github.com/shivang-raj/FitMate
November 2026

Abstract

FitMate is a fitness and workout trainer that runs on **Android**, **iOS** and **desktop**, with a small **Python FastAPI** service that finds exercise tutorial videos. A user enters their details once (age, height, weight, gender, activity level, goal and experience) and the app builds a seven-day workout plan, records completed workouts and the calories burned, works out health numbers such as BMI, BMR and a daily calorie target, gives personalised tips and sends daily reminders. There is no login: every piece of personal data stays on the user's device.

The project was built for the Object-Oriented Programming course, so the main aim was not just a working app but a clean object-oriented design. Every core OOP concept — classes and objects, encapsulation, inheritance, polymorphism, abstraction, interfaces, generics and composition — is used inside a real feature rather than a toy example. Six design patterns (Strategy, Factory, Singleton, Repository, Observer and Dependency Injection) keep the code easy to extend: adding a new fitness goal, health metric or suggestion rule means adding one new class without editing the screens.

The Android app is written in Kotlin with Jetpack Compose, the iOS app in Swift with SwiftUI, and the desktop app in Kotlin with Compose Desktop. Together the four parts contain about 8,800 lines of source code. Fourteen Android unit tests and five backend tests check the core logic.

Keywords

Object-oriented programming · Kotlin · Jetpack Compose · Swift · FastAPI · design patterns · fitness tracking · health metrics

Contents

Abstract 2
1 **Introduction** 4
2 **Features** 5
3 **Technology Stack** 6
4 **System Architecture** 7
5 **Object-Oriented Concepts** 9
6 **Design Patterns** 14
7 **UML Diagrams** 17
8 **Application Screenshots** 19
9 **Testing** 23
10 **Installation and Running** 24
11 **Privacy and Security** 25
12 **Conclusion and Future Work** 26
References 27

1 Introduction

1.1 Background

Many students and young adults want to exercise regularly but do not know where to start. Gym trainers cost money, generic online plans ignore a person's goal and fitness level, and most fitness apps ask for an account, show ads, or upload personal health data to a server. Beginners also struggle with form: doing a push-up or squat the wrong way can cause injury.

FitMate tries to solve these problems with one simple app. It asks for a few details, builds a plan that matches the user's goal, explains each exercise, plays a short tutorial video, and keeps track of progress — all without an account.

1.2 Problem statement

> Design and build a fitness application that creates a personalised weekly workout plan, tracks workouts and calories, calculates key health numbers and reminds the user to stay active — and use the project to demonstrate every major object-oriented programming concept and common design patterns in real, working code.

1.3 Objectives

1. Collect the user's profile once and **reject invalid values** (for example, an age of 5 or a weight of 500 kg).
2. Generate a **seven-day plan** (Day 1 to Day 7) that depends on the user's goal and experience level.
3. Provide an **exercise library** with clear instructions, grouped by type.
4. Let the user **log workouts**, and estimate calories burned with a standard MET-based formula.
5. Show **progress**: streak, seven-day chart, weekly minutes and calories.
6. Calculate **health numbers**: BMI, healthy weight range, BMR, daily calories, protein and water.
7. Send **daily reminders** for workouts, water, meals, stretching and sleep.
8. Find and play the **best tutorial video** for each exercise.
9. Apply all core **OOP concepts and design patterns** in a way that can be explained and tested.

1.4 Scope

Table 1.1 What the project covers

In scope	Out of scope
Android, iOS and desktop apps with the same core design	User accounts, login or cloud sync
Goal-based weekly plans for four goals and three levels	Diet plans or meal tracking
Local storage of profile, workout logs and reminders	Wearables and step counters
Tutorial search through a small FastAPI backend	Medical advice — health numbers are general guidance only

2 Features

The table below lists FitMate's main features, what each one does for the user, and the classes behind it. The numbers in the strip are taken from the source code.

23	4	4	6	6	5
exercises	exercise types	fitness goals	health metrics	suggestion rules	reminder types

Table 2.1 Main features and the classes that implement them

Feature	What it does	Main classes
Profile	Stores name, age, height, weight, gender, activity level, goal and experience. Bad values are rejected with a clear message.	`UserProfile`
Weekly plan	Builds Day 1 to Day 7 for the chosen goal. Each day has a clear name, such as “Leg Day”, and Sunday is a rest day.	`PlanStrategy`, `WorkoutPlanFactory`, `WorkoutPlan`
Exercise library	23 exercises in four groups — Strength, Cardio, HIIT and Flexibility — each with sets, reps or time and a form tip.	`ExerciseLibrary`, `Exercise`
Workout log	Records a finished exercise with its duration; calories are calculated automatically.	`WorkoutLog`, `Repository<T>`
Progress	Shows the current streak, a seven-day calorie chart, workouts and minutes this week.	`WorkoutStats`
Health	BMI calculator with category, healthy weight range, BMR, daily calories, protein and water.	`HealthMetric` + 6 metrics
Smart tips	Personal suggestions, sorted by priority, such as “Time to get moving” after two rest days.	`SuggestionEngine`, `SuggestionRule`
Tutorials	Finds and plays the best YouTube video for an exercise inside the app.	`TutorialApiClient`, FastAPI backend
Reminders	Daily notifications for workout, water, meal, stretch and sleep; restored after a phone restart.	`Reminder`, `ReminderScheduler`

3 Technology Stack

FitMate has four parts. The Android and desktop apps share the same Kotlin model and planner classes; the iOS app follows the same design in Swift. The backend is only used to search for tutorial videos, so the apps work offline for everything else.

Table 3.1 Platforms and tools

Part	Language	UI / framework	Notes
Android app	Kotlin	Jetpack Compose	Min SDK 26, target SDK 36; alarms and notifications
iOS app	Swift	SwiftUI	iOS 16 or later; same classes as Android
Desktop app	Kotlin	Compose Desktop	JCEF browser for playing tutorials
Backend	Python 3	FastAPI, Uvicorn	YouTube Data API v3, SQLite for feedback
Build and tools	—	Gradle, Xcode, pip	Git and GitHub for version control

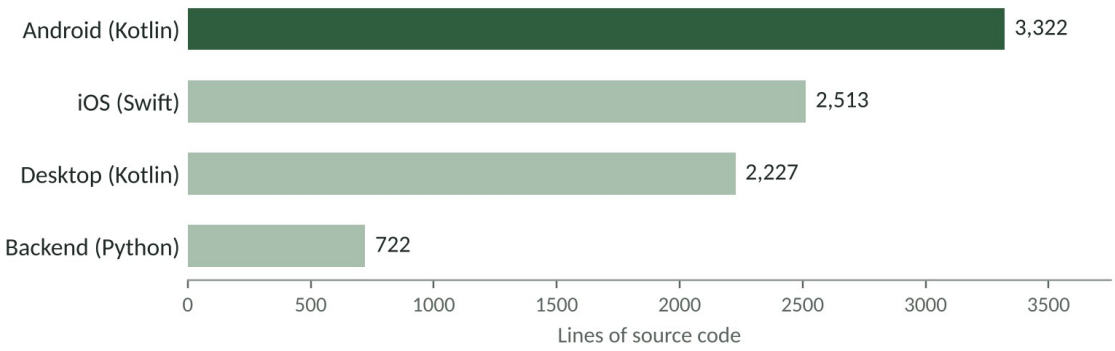


Figure 3.1 Size of each part of the project, in lines of source code

4 System Architecture

4.1 Layered design

The Android app is split into layers. Screens only draw the interface and pass user actions to `FitnessViewModel`. The view model holds the app's state and calls the logic classes, the storage classes and the tutorial client. Because the screens never talk to storage or alarms directly, either of these can be replaced without touching the UI.

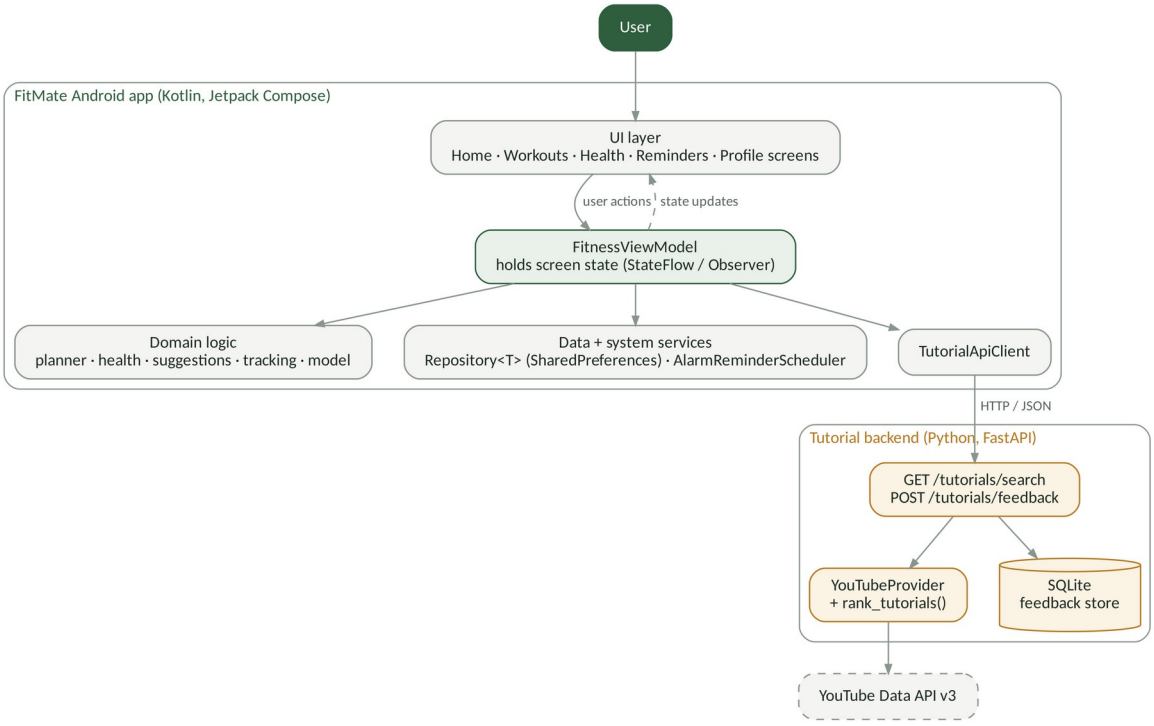


Figure 4.1 Architecture of the Android app and the tutorial backend

Table 4.1 Packages and their responsibilities

Package	Responsibility	Key classes
model	Domain data	`Exercise` + 4 subclasses, `UserProfile`, `WorkoutPlan`, `WorkoutLog`, `Reminder`, enums
planner	Builds the weekly plan	`PlanStrategy`, `BasePlanStrategy`, 4 strategies, `WorkoutPlanFactory`
health	Health calculations	`HealthMetric` + 6 metrics
suggestions	Personal tips	`SuggestionEngine`, `SuggestionRule` + 6 rules
tracking	Weekly statistics	`WorkoutStats`
data	Saving and loading	`Repository<T>`, `PrefsRepository<T>`, `JsonMapper<T>`
reminders	Alarms and notifications	`ReminderScheduler`, `AlarmReminderScheduler`
viewmodel / ui	Screen state and screens	`FitnessViewModel`, Compose screens

4.2 How a user moves through the app

Each step of the user's journey is handled by its own class. This is the single-responsibility principle in practice.

Table 4.2 User journey and the class in charge of each step

Step	What happens	Class in charge
1. Create profile	User enters details; each value is checked before it is saved.	`UserProfile`
2. Weekly plan	The factory picks the plan class for the goal and builds seven days.	`PlanStrategy`
3. Log a workout	User logs an exercise; calories are worked out from its own formula.	`WorkoutLog`
4. Health and tips	Metrics and rules run on the profile and the workout history.	`HealthMetric`
5. Reminders	Daily alarms are scheduled and shown as notifications.	`ReminderScheduler`

4.3 Tutorial search

When the user taps **Watch tutorial**, the app sends the exercise name to the backend. The backend searches YouTube, removes videos that cannot be embedded or are not relevant, scores the rest and returns the best one, which then plays inside the app.

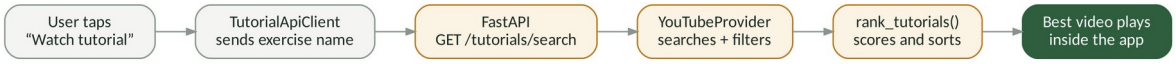


Figure 4.2 Steps from tapping "Watch tutorial" to playing a video

`rank_tutorials()` gives each video a score between 0 and 1 using four weighted signals. Videos with a relevance below **0.30**, videos that cannot be embedded, and videos the user has rejected before are skipped. If a signal is missing (for example, a hidden like count), the remaining weights are rescaled so the ranking still works.

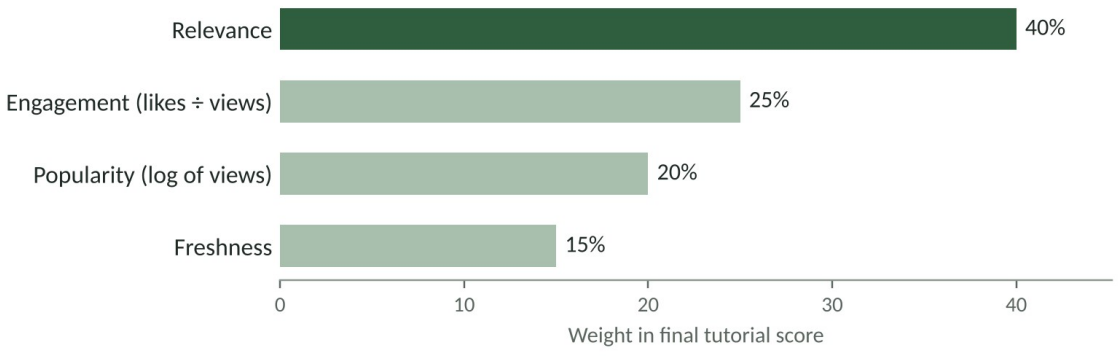


Figure 4.3 Weights used to score a tutorial video

5 Object-Oriented Concepts

This chapter shows where each OOP concept is used in FitMate. All code is taken from the Android app; the desktop app reuses the same classes and the iOS app mirrors them in Swift.

Table 5.1 Summary of OOP concepts in FitMate

Concept	Where it is used	Class
Class and object	Each workout the user logs is a new object	`WorkoutLog`
Encapsulation	The profile checks its own values before saving them	`UserProfile`
Inheritance	All exercise types reuse one parent class	`Exercise`
Polymorphism	Each exercise type counts time and calories its own way	`caloriesBurned()`
Abstraction	Screens use a health metric without knowing its formula	`HealthMetric`
Interface	Every plan class has the same `buildPlan()` method	`PlanStrategy`
Generics	One storage class saves both workouts and reminders	`Repository<T>`
Composition	A plan has 7 sessions; each session has exercises	`WorkoutPlan`

5.1 Classes and objects

A class is a blueprint and an object is one instance of it. FitMate has about 67 classes, interfaces, enums and objects. `WorkoutLog` is a Kotlin `data class`, so the compiler generates `equals()`, `hashCode()`, `toString()` and `copy()`. Its companion function `from()` builds a log from any kind of exercise.

model/WorkoutLog.kt

```
data class WorkoutLog(
    override val id: String = UUID.randomUUID().toString(),
    val exerciseName: String, val category: ExerciseCategory,
    val minutes: Int, val calories: Double,
    val timestamp: Long = System.currentTimeMillis()
) : Identifiable, Describable {
    companion object {
        // Factory method: works for ANY Exercise subclass
        fun from(exercise: Exercise, minutes: Int, weightKg: Double) =
            WorkoutLog(exerciseName = exercise.name, category = exercise.category,
                minutes = minutes,
                calories = exercise.caloriesBurned(weightKg, minutes))
    }
}
```

5.2 Encapsulation

`UserProfile` never stores an impossible value. Its numeric fields can only be changed through setters that check the range first and throw an `IllegalArgumentException` if the value is wrong. The constructor values also go through the same setters in the `init` block, so a profile is valid both when it is created and every time it is changed. BMI is a read-only computed property, so it can never be out of date.

model/UserProfile.kt

```
var age: Int = 0
    set(value) {
        require(value in MIN_AGE..MAX_AGE) { "Age must be $MIN_AGE-$MAX_AGE" }
        field = value
    }

init {   // constructor values go through the same setters
    this.name = name; this.age = age
    this.heightCm = heightCm; this.weightKg = weightKg
}

val bmi: Double
    get() = weightKg / ((heightCm / 100) * (heightCm / 100))
```

Table 5.2 Validation rules in UserProfile

Field	Allowed range	Message shown if invalid
Name	Not blank	Name can't be empty
Age	10 – 100 years	Age must be between 10 and 100
Height	80 – 250 cm	Height must be between 80 and 250 cm
Weight	20 – 300 kg	Weight must be between 20 and 300 kg

5.3 Inheritance and polymorphism

`Exercise` is an abstract class that holds the data every exercise shares: id, name, description, difficulty and MET value (how many times resting energy the activity burns). It cannot be created directly. Four subclasses add their own fields and their own way of working out the time. `HiitExercise` extends `CardioExercise`, which is **multi-level inheritance**.

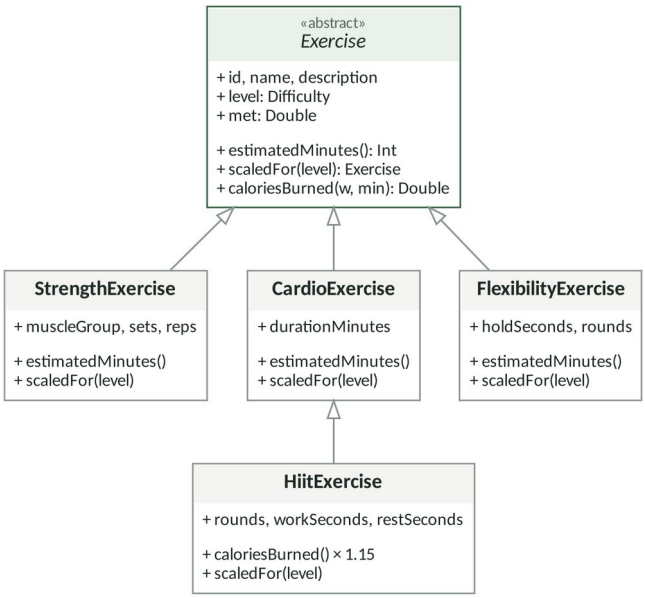


Figure 5.1 Exercise class hierarchy

The app always treats exercises as plain `Exercise` objects. When it calls `estimatedMinutes()` or `caloriesBurned()`, Kotlin runs the version that belongs to the real subclass at run time. This is **runtime polymorphism** (method overriding).

Table 5.3 How each exercise type works out its time and calories

Type	Estimated time	Calories	Example
Strength	4 s per rep + 90 s rest between sets	MET × 3.5 × kg ÷ 200 × min	Push-ups, 3 × 12 → 6 min
Cardio	Fixed duration	MET × 3.5 × kg ÷ 200 × min	Running, 20 min
HIIT	rounds × (work + rest)	Parent formula × 1.15 (afterburn)	Mountain climbers, 8 rounds → 8 min
Flexibility	hold × rounds × 2 sides	MET × 3.5 × kg ÷ 200 × min	Cobra stretch, 20 s × 4

model/Exercise.kt — HiitExercise reuses its parent's formula with super

```
override fun caloriesBurned(weightKg: Double, minutes: Int): Double =
    super.caloriesBurned(weightKg, minutes) * AFTERBURN_BONUS   // 1.15
```

> **Worked example**
>
> For a 78 kg user doing 8 minutes of mountain climbers (MET 8.0): 8.0 × 3.5 × 78 ÷ 200 × 8 = 87.4 kcal; with the 15% HIIT bonus this becomes **≈ 100 kcal**, which is exactly what the Android app shows in Figure 8.1.

5.4 Abstraction and interfaces

`HealthMetric` is an interface with `calculate()`, `interpret()` and a default `formatted()` method. The Health screen loops over a list of six metrics and never needs to know which formula it is drawing. A new metric can be added by writing one class and adding it to the list.

health/HealthMetric.kt

```
interface HealthMetric {
    val name: String
    fun calculate(profile: UserProfile): Double
    fun interpret(profile: UserProfile): String
    fun formatted(profile: UserProfile): String = "%.1f".format(calculate(profile))
}
class BmiMetric : HealthMetric { ... }
```

Table 5.4 The six health metrics and a worked example (female, 20 years, 166 cm, 68 kg, sedentary, “Stay fit”)

Metric	Formula	Result
Body Mass Index	weight ÷ height²	24.7 — Healthy
Healthy weight range	BMI 18.5 to 24.9 at the user's height	51 – 69 kg
Basal Metabolic Rate	Mifflin–St Jeor: 10w + 6.25h − 5a + s (s = +5 male, −161 female)	1,457 kcal

Metric	Formula	Result
Daily calories	BMR × activity factor (1.2 – 1.9) + goal change (−500 / +300 / 0)	1,748 kcal
Daily protein	1.2 – 1.8 g per kg, depending on goal	82 g
Daily water	0.033 L per kg, + 0.5 L if very active	2.2 L

The BMI value in this example matches the desktop Profile screen (Figure 8.6). `DailyCaloriesMetric` also shows **composition**: it holds a `BmrMetric` object and reuses its formula instead of copying it.

5.5 Generics

Workout logs and reminders are both saved as JSON in the phone's `SharedPreferences`. Instead of writing two storage classes, FitMate uses one generic `Repository<T>`. The type parameter is bounded by `Identifiable`, so every stored item is guaranteed to have an id. The type-specific JSON conversion is passed in as a `JsonMapper<T>`.

data/Repository.kt

```
interface Repository<T : Identifiable> {
    fun getAll(): List<T>
    fun save(item: T)          // insert, or replace the item with the same id
    fun delete(id: String)
}

// AppContainer: one generic class, two different types
val workoutRepository: Repository<WorkoutLog> =
    PrefsRepository(prefs, "workout_logs", WorkoutLogMapper)
val reminderRepository: Repository<Reminder> =
    PrefsRepository(prefs, "reminders", ReminderMapper)
```

Generics are also used in `ExerciseLibrary.ofType<T>()`. Calling `ofType<CardioExercise>()` returns cardio **and** HIIT exercises, because a `HiitExercise` is a `CardioExercise`.

5.6 Composition

A `WorkoutPlan` **has** seven `WorkoutSession` objects, and each session **has** a list of exercises. The plan's totals are built from its parts: weekly minutes is the sum of each session's minutes, and each session asks every exercise for its own calories.

model/WorkoutPlan.kt

```
class WorkoutPlan(val name: String, val goal: FitnessGoal,
                  val level: Difficulty, val sessions: List<WorkoutSession>) {
    val activeDays: Int    get() = sessions.count { !it.isRestDay }
    val weeklyMinutes: Int get() = sessions.sumOf { it.totalMinutes() }
}
```

5.7 Other features of the language

Table 5.5 Additional OOP features used

Feature	Use in FitMate
Enums as classes	`ActivityLevel` carries its multiplier; `BmiCategory.from(bmi)` picks the right category.
Companion objects (static members)	Validation limits such as `MIN_AGE` and factory methods like `WorkoutLog.from()`.
Operator overloading	`ExerciseLibrary["squats"]` looks up an exercise by id.
Immutability	`Reminder` is a data class; changes are made with `copy()`.
Visibility modifiers	`protected` helpers in `BasePlanStrategy`; `private` fields in repositories.

6 Design Patterns

Design patterns are reusable solutions to common design problems. FitMate uses six of them, plus the Template Method pattern inside the planner.

Table 6.1 Design patterns used in FitMate

Pattern	Where	Why it helps
Strategy	One plan class per goal	The plan algorithm can change without changing the code that uses it.
Factory	`WorkoutPlanFactory`	Callers ask for a plan and never need to know which class builds it.
Template Method	`BasePlanStrategy`	Fixes the steps once; subclasses only fill in the weekly schedule.
Singleton	`ExerciseLibrary` (`object`)	The whole app reads from one shared exercise catalogue.
Repository	`Repository<T>`	Hides how and where data is stored.
Observer	`StateFlow` in `FitnessViewModel`	Screens redraw automatically when the data changes.
Dependency Injection	`AppContainer`	Creates shared objects once and hands them out as interfaces.

6.1 Strategy and Factory: building the weekly plan

Each fitness goal has its own plan class. All four extend `BasePlanStrategy`, which implements the `PlanStrategy` interface. `buildPlan()` is marked `final`: it always gets the schedule, scales each exercise to the user's level and numbers the days 1 to 7. Subclasses can only decide which exercises go on which day.

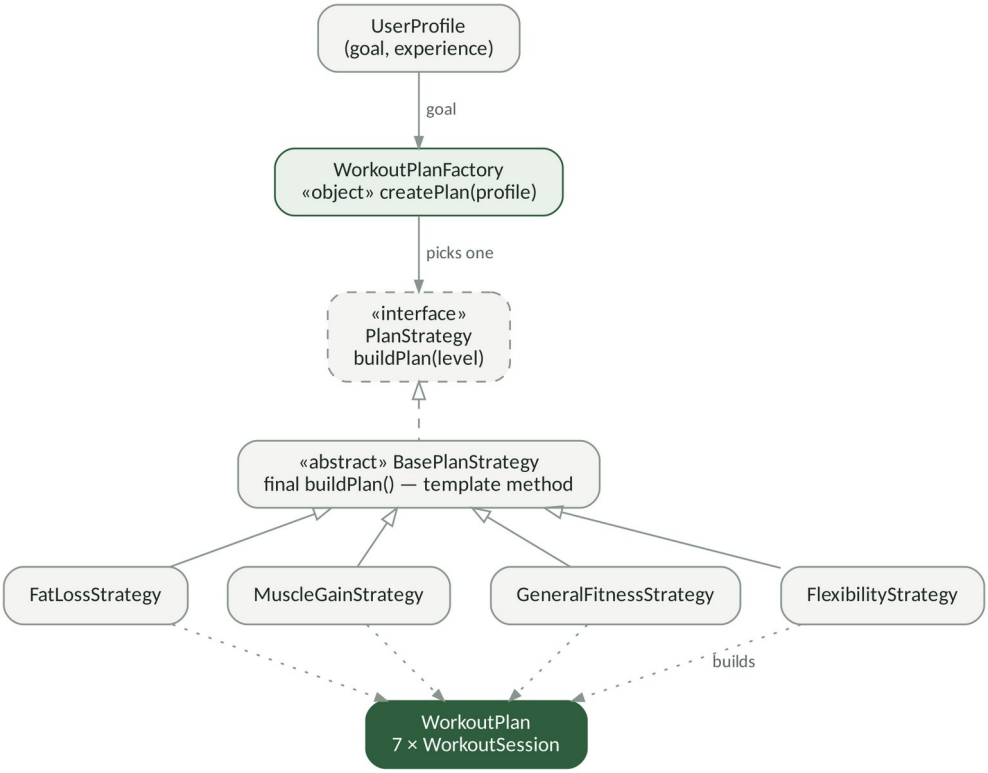


Figure 6.1 Strategy and Factory patterns in the planner

planner/PlanStrategy.kt

```
object WorkoutPlanFactory {
    fun strategyFor(goal: FitnessGoal): PlanStrategy = when (goal) {
        LOSE_WEIGHT         -> FatLossStrategy()
        BUILD_MUSCLE        -> MuscleGainStrategy()
        STAY_FIT            -> GeneralFitnessStrategy()
        IMPROVE_FLEXIBILITY -> FlexibilityStrategy()
    }
    fun createPlan(profile: UserProfile): WorkoutPlan =
        strategyFor(profile.goal).buildPlan(profile.experience)
}
```

Table 6.2 The four plans generated by the strategies

Goal	Plan name	Week (Day 1 → Day 6, Day 7 is rest)
Lose weight	Fat Burn Program	Full-Body HIIT · Cardio · Leg · Upper-Body · Tabata · Recovery Yoga
Build muscle	Push / Pull / Legs	Chest · Back · Leg · Shoulder · Full-Body · Mobility
Stay fit	Balanced Fitness	Full-Body · Cardio · Yoga · HIIT · Core & Cardio · Legs & Walk
Flexibility	Mobility & Yoga	Yoga · Hips · Full Stretch · Core · Yoga Flow · Stretch & Cycle

The experience level then changes the volume. An intermediate user gets one extra set on strength exercises and an advanced user gets two; cardio time is multiplied by 0.8, 1.0 or 1.3.

6.2 Open/closed suggestion rules

`SuggestionEngine` holds a list of `SuggestionRule` objects. Each rule looks at the profile and workout stats and either returns a tip or `null`. The engine collects the tips and sorts them by priority. A new tip is added by writing one new rule class — the engine itself never changes. This follows the **open/closed principle**: open for extension, closed for modification.

Table 6.3 Suggestion rules

Rule	When it fires	Example tip
InactivityRule	No workout yet, or 2+ days since the last one	“Time to get moving”
BmiRule	Always; advice depends on the BMI category	“Healthy BMI”
GoalRule	Always; advice depends on the goal	“Protein + progressive overload”
WeeklyTargetRule	Compares weekly minutes with the 150-minute WHO guideline	“30 min to your weekly goal”
RecoveryRule	Trained on 6 or more of the last 7 days	“Schedule a rest day”
HydrationRule	Always	“Stay hydrated” with the daily water target

7 UML Diagrams

7.1 Use case diagram

The user is the main actor. Two external systems support the app: the tutorial backend (with YouTube) and the phone's operating system, which fires alarms and restarts them after the phone reboots.

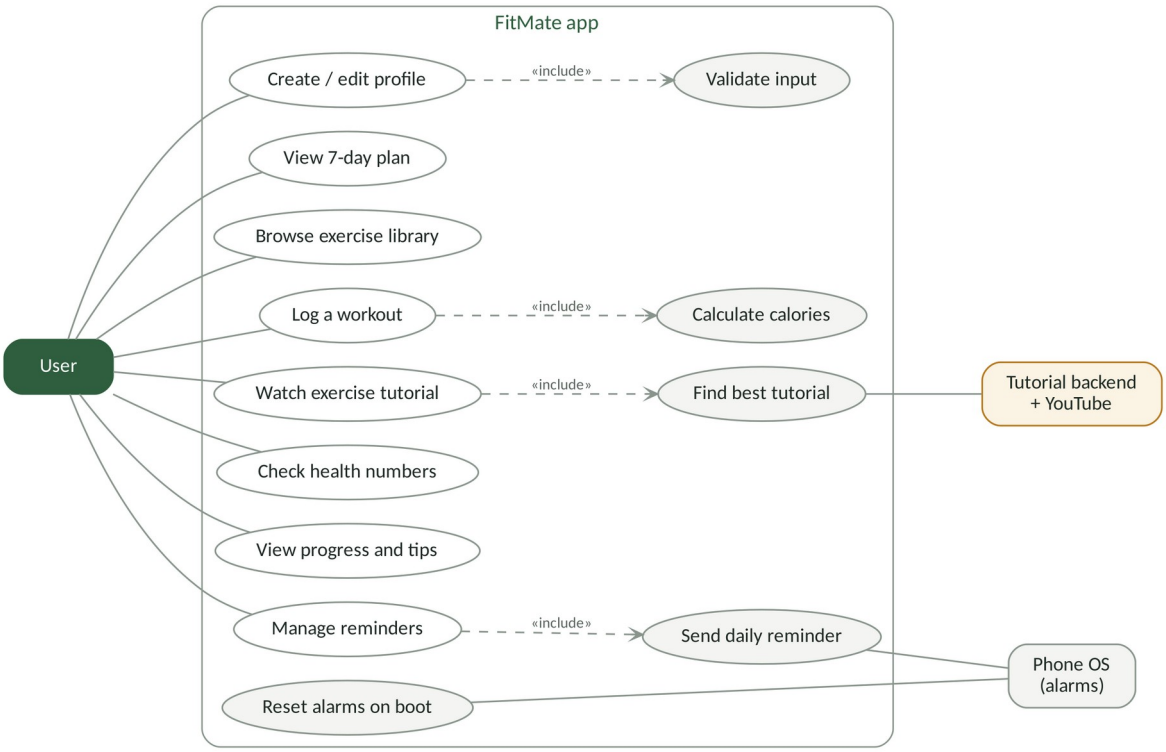


Figure 7.1 Use case diagram of FitMate

7.2 Class diagram

The simplified class diagram below shows the main classes and how they relate. `FitnessViewModel` sits at the centre and depends on interfaces, not concrete classes. Filled diamonds show composition, hollow diamonds show aggregation and hollow triangles show inheritance or interface implementation. The full diagram, with every field and method, is in the repository at `docs/oop/class_diagram.png`.

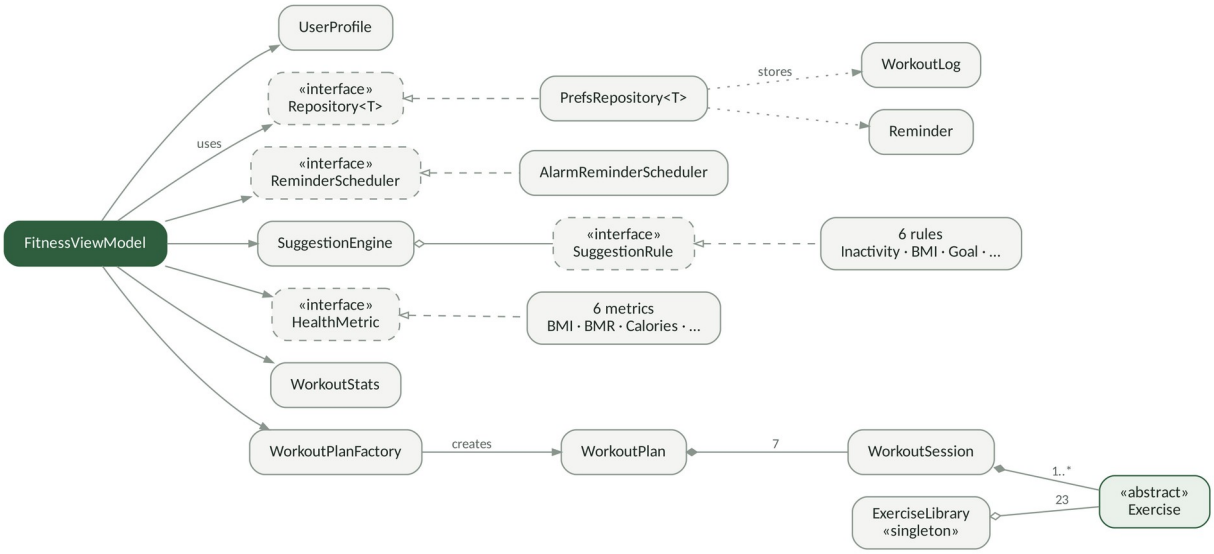


Figure 7.2 Simplified class diagram of the Android app

Table 7.1 How to read the relationships in the diagram

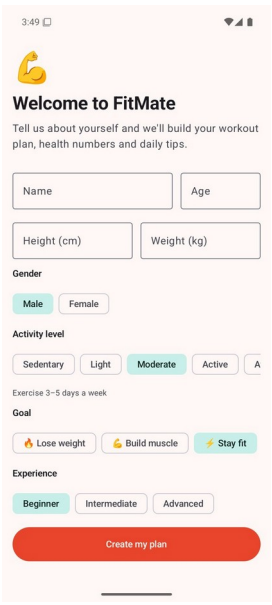
Relationship	Line in the diagram	Example in FitMate
Association (uses)	Solid line with an open arrow	`FitnessViewModel` uses `SuggestionEngine`
Inheritance / implementation	Hollow triangle (dashed for an interface)	`PrefsRepository<T>` implements `Repository<T>`
Composition	Filled diamond on the owner	`WorkoutPlan` owns 7 `WorkoutSession` objects
Aggregation	Hollow diamond on the container	`ExerciseLibrary` holds 23 `Exercise` objects
Dependency	Dotted line with an arrow	`PrefsRepository<T>` stores `WorkoutLog` and `Reminder`

8 Application Screenshots

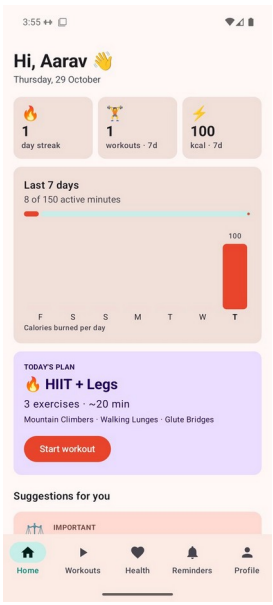
8.1 Android app

The Android app has five tabs — Home, Workouts, Health, Reminders and Profile — and a one-time onboarding screen. These screenshots are taken from the project repository.

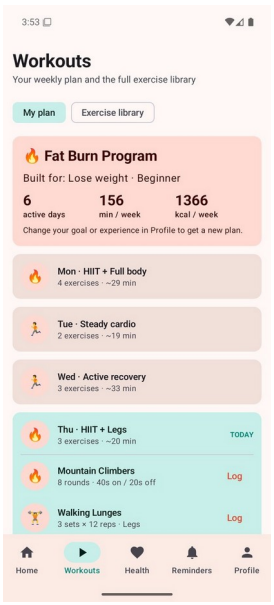
Figure 8.1 Android app screens



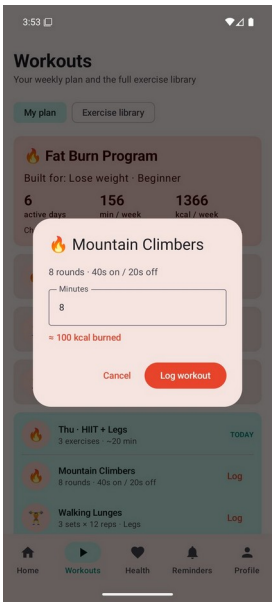
Onboarding
Profile is entered once



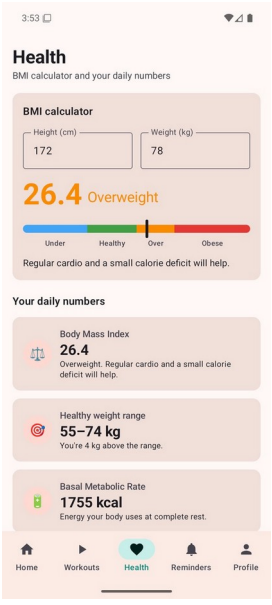
Home
Streak, 7-day chart, today's plan



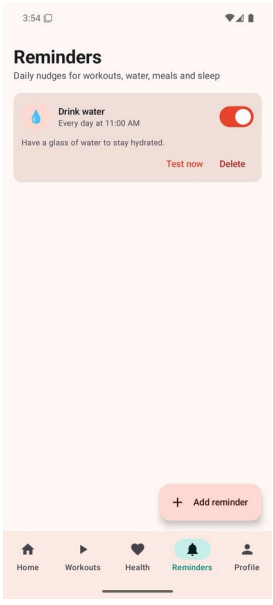
Workouts
Goal-based weekly plan



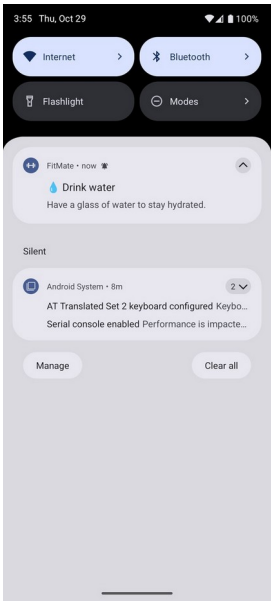
Log workout
Calories worked out for you



Health
BMI calculator and daily numbers



Reminders
Daily nudges with a test button



Notification
Reminder shown by the system

8.2 Desktop app

The desktop app uses the same model and planner classes as Android, with a sidebar layout that suits a larger screen. Tutorials play inside the app through an embedded browser.

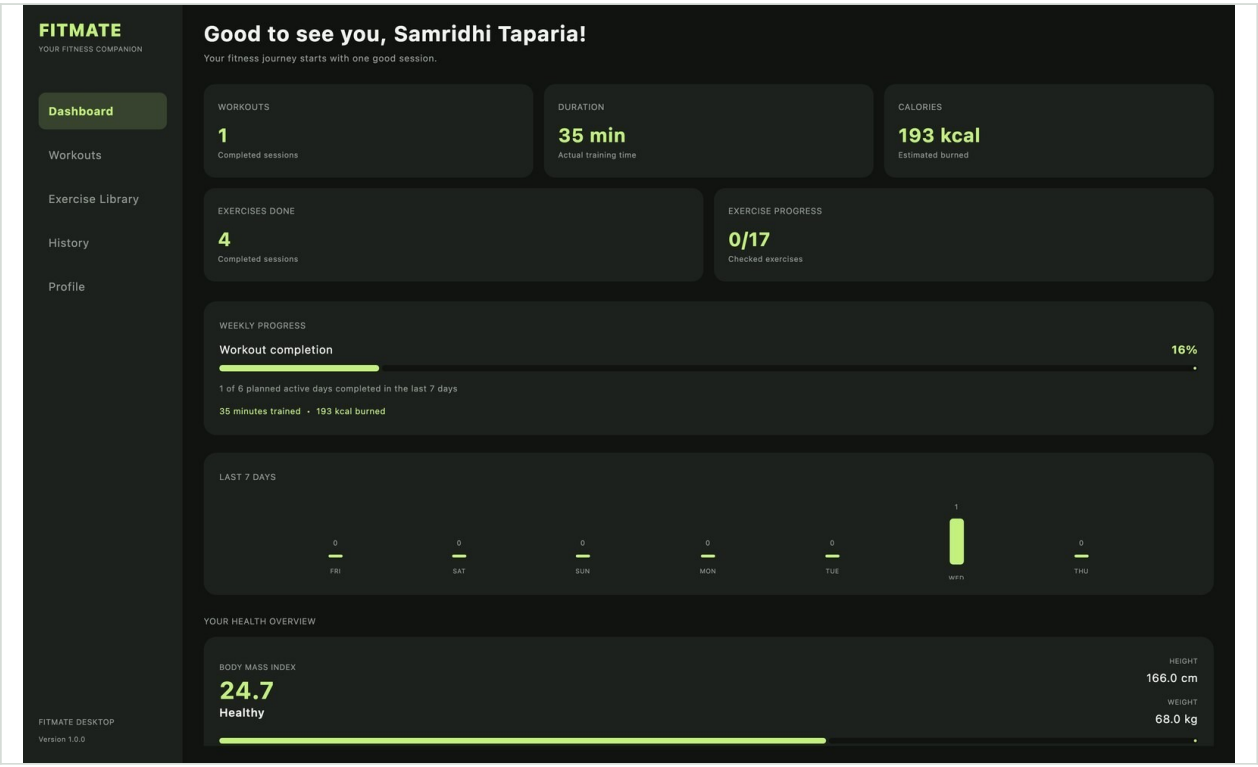


Figure 8.2 Desktop dashboard: workouts, duration, calories, weekly progress and BMI

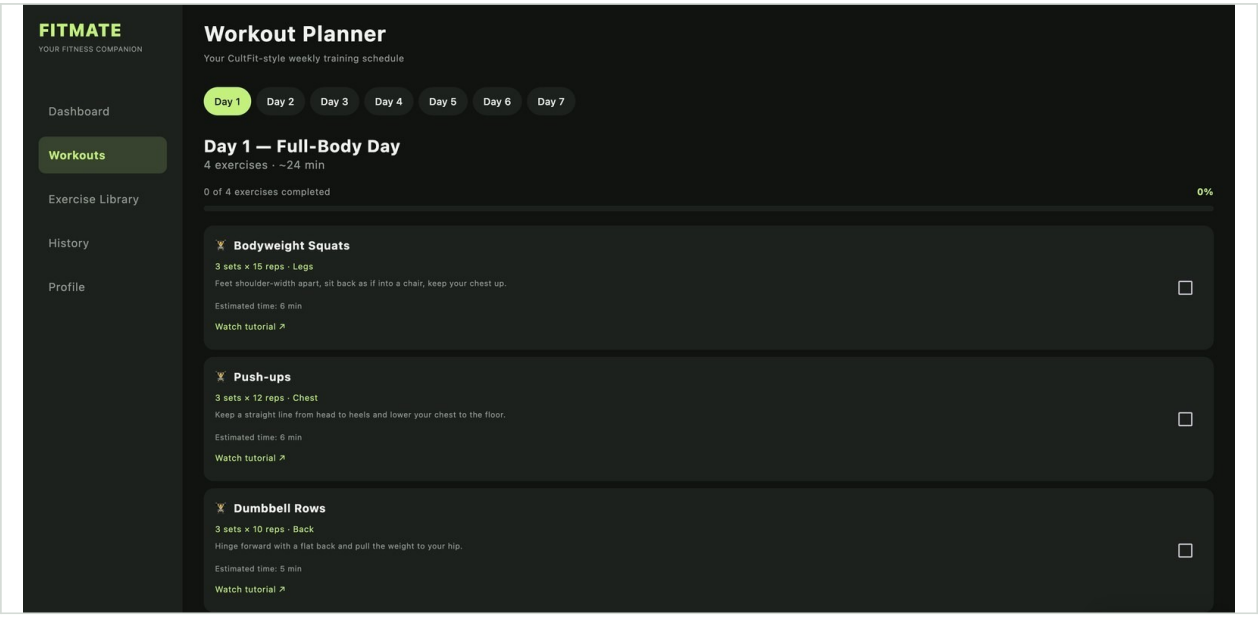


Figure 8.3 Desktop workout planner: Day 1 — Full-Body Day with checkboxes for each exercise

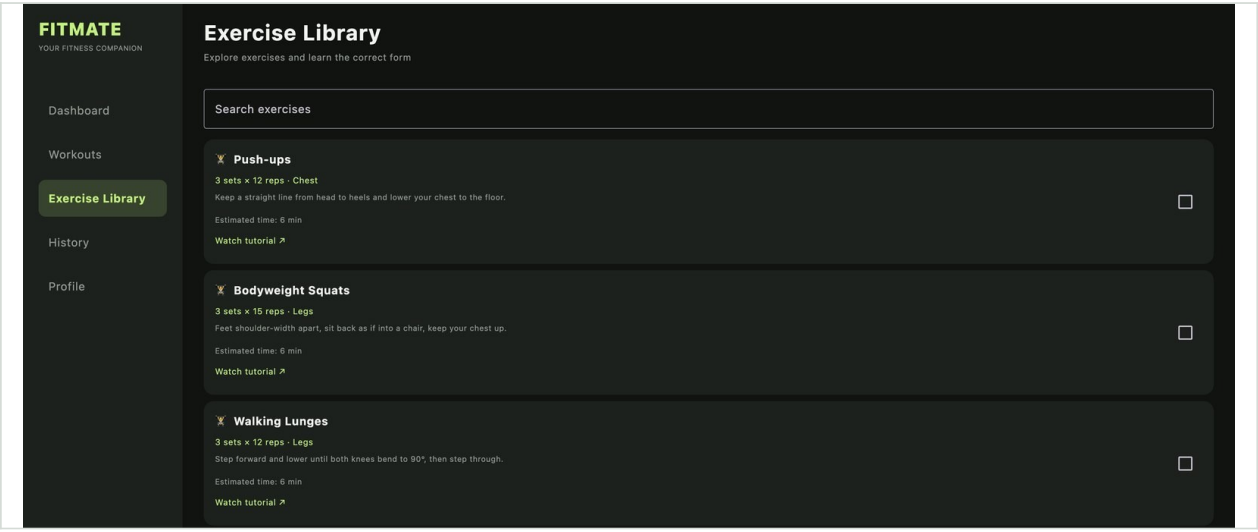


Figure 8.4 Desktop exercise library with search, form tips and tutorial links

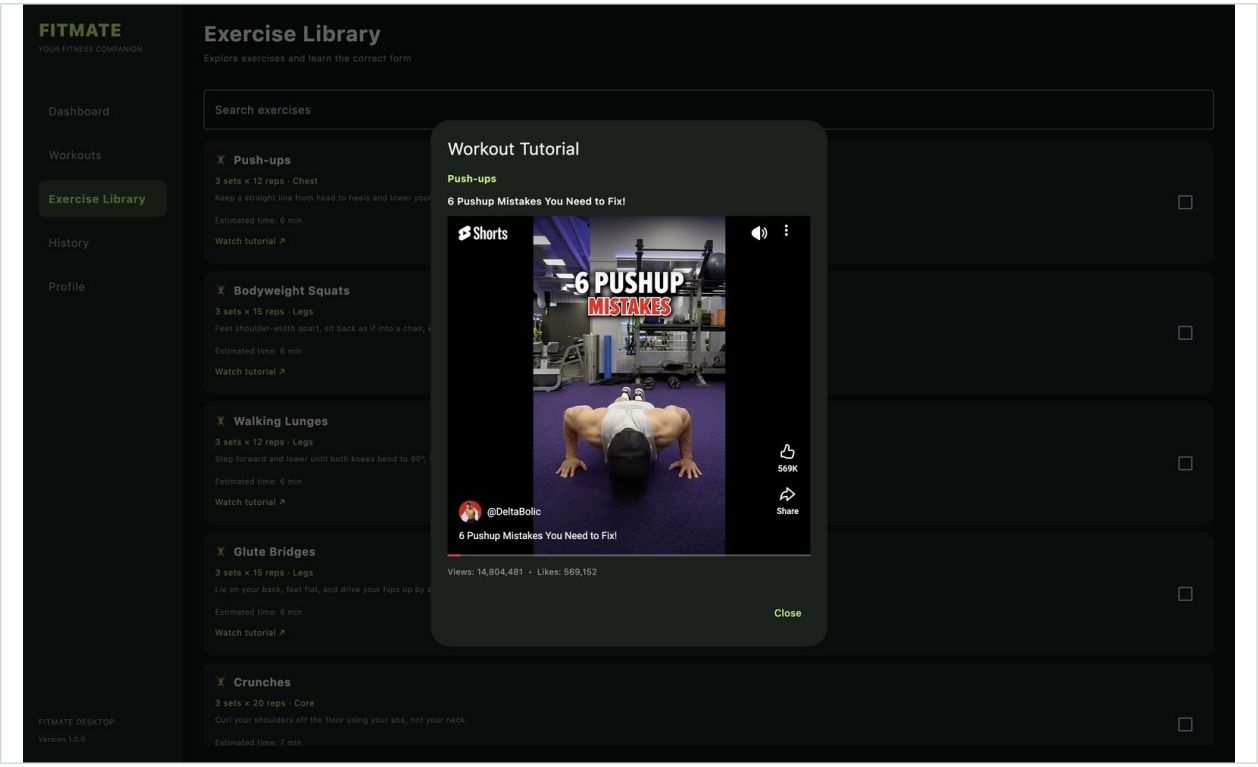


Figure 8.5 Tutorial video playing inside the desktop app, chosen by the backend's ranking

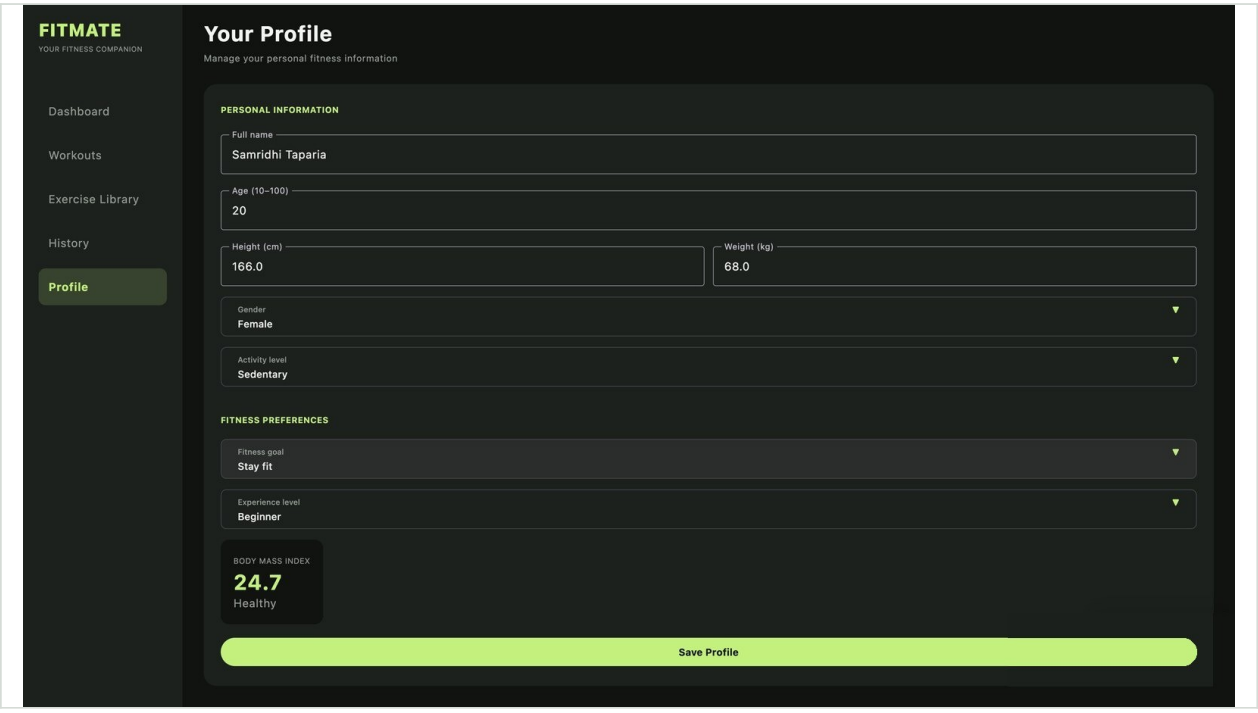


Figure 8.6 Desktop profile screen with live BMI

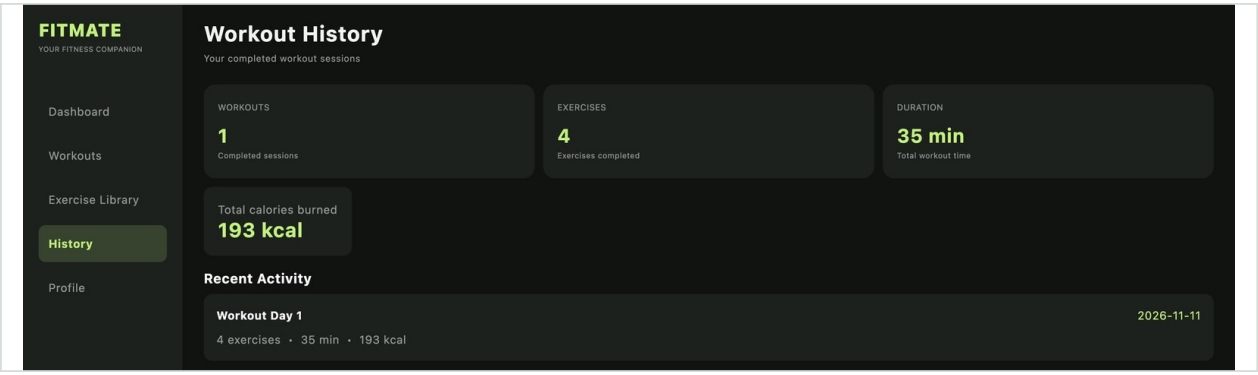


Figure 8.7 Desktop workout history

9 Testing

Because the logic lives in small classes with no Android code inside them, it can be tested with plain JUnit tests that run on a computer in seconds. The backend ranking is tested with pytest.

Running the tests

```
cd android && ./gradlew testDebugUnitTest     # Android unit tests
cd backend && pytest                          # backend tests
```

Table 9.1 Android unit tests (DomainTest.kt)

#	Test	What it checks
1	bmiIsComputedFromHeightAndWeight	BMI formula gives the right value
2	impossibleWeightIsRejected	Constructor throws for an out-of-range weight
3	setterAlsoValidates	Changing a field later is also validated
4	hiitIsACardioExerciseAndBurnsMore	Inheritance and the 15% afterburn override
5	scaledForKeepsTheSubclassAndAddsVolume	Scaling keeps the subclass type and adds sets
6	ofTypeUsesTheClassHierarchy	Generic filter includes HIIT under cardio
7	everyHealthMetricProducesText	All six metrics return a value and an explanation
8	factoryPicksTheStrategyForTheGoal	Factory returns the right strategy for each goal
9	everyGoalGetsASevenDayPlan	Every plan has exactly seven sessions
10	streakCountsConsecutiveDays	Streak counting in WorkoutStats
11	newUserIsNudgedToLogFirstWorkout…	First-workout tip appears; tips are sorted
12	everyExerciseHasAYouTubeLink	No exercise is missing a tutorial link
13	scaledExerciseKeepsItsVideoLink	Scaling does not lose the video link
14	planIsNumberedDay1ToDay7WithClearDayNames	Days are numbered and named correctly

Table 9.2 Backend tests (test_ranking.py)

#	Test	What it checks
1	test_best_tutorial_is_ranked_first	The highest-scoring video comes first
2	test_non_embeddable_tutorial_is_excluded	Videos that cannot play in the app are removed
3	test_rejected_tutorial_is_skipped	Videos the user disliked are not shown again
4	test_missing_like_count_does_not_break_ranking	Weights are rescaled when a signal is missing
5	test_all_rejected_returns_empty_list	Empty result is handled safely

10 Installation and Running

Table 10.1 How to build and run each part

Part	Requirements	Steps
Android	Android Studio, JDK	Open the `android` folder, let Gradle sync, press Run. Debug APK: `./gradlew assembleDebug`.
iOS	macOS, Xcode	Open `ios/FitMate.xcodeproj`, choose an iOS 16+ simulator, press Run.
Desktop	JDK, Gradle	Run from the `desktop` folder. The first launch downloads dependencies and the JCEF runtime.
Backend	Python 3, YouTube API key	Create a virtual environment, `pip install -r requirements.txt`, put the key in `backend/.env`, start with `uvicorn app.main:app --reload`.

The backend runs at `http://127.0.0.1:8000`, with interactive API documentation at `/docs`. An Android emulator reaches it at `10.0.2.2`; a real phone needs the computer's local IP address on the same Wi-Fi network.

Table 10.2 Backend API endpoints

Method	Endpoint	Purpose
GET	`/tutorials/search?exercise=…`	Search YouTube, rank the results and return the best tutorial
POST	`/tutorials/feedback`	Save a like or dislike so rejected videos are skipped next time
GET	`/health`	Check that the server is running

11 Privacy and Security

- **No login and no cloud.** The profile, workout history and reminders are stored only on the device.
- **Only the exercise name leaves the phone** when a tutorial is requested — never personal or health data.
- **API keys are kept out of the code.** The YouTube key lives in `backend/.env`, which is listed in `.gitignore`.
- **Validated input** on both sides: `UserProfile` checks every value, and FastAPI checks every request parameter with Pydantic.
- For a public deployment the backend should be served over **HTTPS**.

> **Note**
> Health numbers in FitMate are general guidance based on standard formulas. They are not medical advice.

12 Conclusion and Future Work

12.1 What we built

1. FitMate for **Android, iOS and desktop**, with a Python FastAPI backend for tutorials.
2. Every main **OOP concept** is used in a real feature, not a separate example.
3. **Strategy and Factory** made the plan code easy to extend: a new goal is one new class.
4. **19 automated tests** check the core logic of the app and the backend.

12.2 What we learned

- Plan the classes before writing the screens.
- Small classes with one job are easier to understand, test and share between platforms.
- Depending on interfaces (`Repository<T>`, `ReminderScheduler`) let us change storage or alarms without touching the UI.

12.3 Future work

Table 12.1 Possible improvements

Idea	Benefit
Optional cloud backup	Move data to a new phone without losing history
Step counter and wearable support	Track daily activity automatically
Progressive overload	Increase sets or reps as the user gets stronger
Meal and nutrition log	Compare calories eaten with the daily target
Pose detection with the camera	Check exercise form in real time

References

[1] FitMate project repository. github.com/shivang-raj/FitMate

[2] Kotlin documentation — Classes, inheritance, interfaces and generics. kotlinlang.org/docs

[3] Android Developers — Jetpack Compose and app architecture guide. developer.android.com

[4] Apple Developer — SwiftUI documentation. developer.apple.com/documentation/swiftui

[5] FastAPI documentation. fastapi.tiangolo.com

[6] YouTube Data API v3 reference. developers.google.com/youtube/v3

[7] Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[8] Mifflin, M. D., St Jeor, S. T. et al. (1990). A new predictive equation for resting energy expenditure in healthy individuals. American Journal of Clinical Nutrition, 51(2), 241–247.

[9] Ainsworth, B. E. et al. (2011). Compendium of Physical Activities: a second update of codes and MET values. Medicine & Science in Sports & Exercise, 43(8), 1575–1581.